

1. Exploration Report

Marabou 프레임워크에서 제공하는 샘플 모델 아키텍처, 데이터셋 및 검증 쿼리를 탐색하고 분석하였다. Netron 을 통해 모델들을 시각화하여 분석하였으며, 쿼리들은 텍스트 파일을 직접 분석하였다.

- Architecture & format

지원 아키텍처: MLP(Fully Connected), CNN, Transformer, BNN(Binary Neural Network)

지원 포맷: .onnx (범용), .nnet (Marabou/Reluplex 전용 검증용), .h5 (Keras/Tensorflow)

비선형 함수인 sigmoid 와 tanh 는 물론 self-attention 기반의 분류 모델들도 포함되어 있는 등 비선형 연산에 대한 선형 근사(Linearization)도 지원한다.

- Dataset & Input Specification/Verification Query

Datasets: MNIST, Fashion-MNIST, ACAS Xu, CIFAR10, CollisionAvoidance, TwinStreams 등에 대한 모델들이 업로드되어 있다.

Input Specification/Verification Query: resources/properties/ 내의 텍스트 파일들에서 입력과 출력의 bound 를 부등식 형태로 확인할 수 있다. ACAS Xu 와 MNIST 에 대한 파일들로 집중되어 있다. 특히 MNIST 에 대해서는 특정 MNIST 이미지에 대해 지정된 범위 내의 적대적 노이즈가 추가되었을 때의 Robustness 를 검증하는 쿼리들이 나열되어 있다.

2. Install Marabou

- issues encountered

- Windows 미지원: Marabou 의 C++ 기반 라이브러리(Boost, OpenBLAS 등)와 Windows 간의 컴파일 호환성 문제로 인해 설치가 불가능하다. WSL2(Windows Subsystem for Linux)를 설치하고 **VS Code** 의 **Remote - WSL** 확장 기능을 사용하여 리눅스 기반 개발 환경으로 전환하여 사용했다.
- Python 버전: Marabou Python API(maraboupy)는 3.8~3.11 버전만을 안정적으로 지원한다. 그러나 Ubuntu 24.04(Noble) 버전의 기본 저장소(apt)에는 3.11 이하 버전이 존재하지 않는다. **deadsnakes PPA(Personal Package Archive)** 저장소를 시스템에 추가하여 구버전 파이썬을 설치했다.

- verification of installation

Maraboupy 공식 documentation 의 testing 탭을 따랐다.

(https://neuralnetworkverification.github.io/Marabou/Setup/1_Testing.html)

3. Running Marabou

1. 모델 설명 (Model Description)

본 실험에서는 MNIST 숫자 이미지(28x28)를 분류하기 위한 SmallCNN 구조를 설계하고 학습했다.

검증 최적화 (Lightweight Design): Marabou 와 같은 정형 검증 도구는 신경망의 모든 연산을 수식으로 변환한다. 따라서 검증 속도와 확장성을 고려하여 채널 수를 최소화(16, 32)하고 파라미터 수를 줄인 경량 모델을 채택했다.

비선형성 제어: 활성화 함수로는 Marabou 가 효율적으로 처리할 수 있는 ReLU 를 사용했다.

출력층 구성 (Softmax 제거): 정형 검증의 목적은 클래스 간 점수의 상대적 크기를 비교하는 것이므로, 수식의 단순화를 위해 Softmax 를 제거한 Logit 값을 직접 출력하도록 설계했다.

성능: 5 Epoch 학습 결과, 테스트 데이터셋 기준 98.83%의 높은 정확도를 달성하여 검증 대상으로서 충분한 신뢰도를 확보했다.

2. Verification Query (검증 쿼리 정의)

입력 제약 (Input Constraints): 원본 이미지 x 에 대해 각 픽셀의 변화폭을 ε 으로 제한하는 L_∞ -ball 영역을 정의한다.

출력 제약 (Output Constraints): 원본 클래스가 label 일 때, 다른 모든 클래스 j 에 대해 $y_{label} > y_j$ 가 성립해야 한다. Marabou 는 반례(Counter-example)를 찾는 도구이므로, 실제 쿼리는 $\exists j \neq label \text{ s.t. } y_j \geq y_{label}$ 조건의 만족 여부를 확인한다.

3. 실행 코드 설명 (Implementation)

작성된 코드 test.py 는 크게 전처리, 제약 조건 생성, 반복 검증의 세 단계로 구성된다.

Marabou.read_onnx(): 학습된 모델을 Marabou 내부 그래프 구조로 로드한다.

setLowerBound / setUpperBound: 784 개의 입력 변수에 대해 이미지 픽셀값 기반의 L_∞ 범위를 할당한다.

Marabou.Equation: GE (Greater than or Equal) 타입을 사용하여 오분류 발생 조건($y_j - y_{label} \geq 0$)을 수식으로 생성하고 네트워크에 추가한다.

Looping & Re-loading: Marabou 객체의 상태 초기화를 위해 각 클래스 검증마다 모델을 재로드하여 독립적인 검증을 수행한다.

4. 결과 및 해석 (Results & Interpretation)

검증 결과, $\epsilon = 0.001$ 범위에서 정답이 아닌 모든 클래스에 대해 UNSAT 판정을 받았다. 즉, 본 모델은 숫자 7 이미지에 대해 매우 미세한 노이즈(± 0.001)가 추가되어도 절대 다른 숫자로 오분류하지 않음을 수학적으로 증명했다. 이는 해당 영역 내에 어떠한 적대적 공격(Adversarial Attack)도 성공할 수 없음을 보장한다.

5. Discussion: Marabou 의 강점과 한계

실제 환경 구축부터 검증까지의 경험을 바탕으로 분석한 Marabou 의 특성은 다음과 같다.

강점 (Strengths)

수학적 보장 (Soundness): 근사치가 아닌 확정적 결과를 제공하여 안전이 직결된 분야에서 강력한 신뢰를 제공한다.

유연한 수식 추가: Equation 클래스를 통해 사용자가 원하는 복잡한 Property 를 자유롭게 정의하여 검증할 수 있다.

한계 (Limitations)

확장성 문제: $\epsilon = 0.01$ 에서는 54,414 개의 변수를 처리하는 도중 연산량이 폭발하여 검증이 제대로 되지 않았다. 레이어 수나 채널 수가 늘어날수록 계산 복잡도가 기하급수적으로 증가하여 검증이 불가능해진다.

지원 포맷 제약: ONNX 포맷을 지원하지만, 특정 Opset 버전이나 연산자 처리에 있어 버전 호환성이 매우 민감하게 작용함을 경험했다.